# 부트로더란 무엇인가

전원에서 커널까지 · QEMU virt 위에서 OpenSBI, U-Boot, Linux로 이어지는 부팅을 직접 따라가며

전원이 들어온 CPU는 정해진 주소의 명령어 하나를 실행할 수 있을 뿐이다. 커널은 아직 저장장치 안에 있고, 메모리에는 아무것도 없으며, 커널이 기대하는 정보(코어 번호, 메모리 크기, 장치 목록)도 없다. 이 간격을 메우는 프로그램이 **부트로더**다. 이 문서는 RISC-V QEMU `virt` 보드에서 부팅 사슬을 단계마다 멈춰 세우고, 각 단계가 무엇을 하고 다음 단계에 무엇을 넘기는지 확인한 기록이다.

2026년 10월 21일 기록. 대상: QEMU 11.1.2 (`-machine virt`), OpenSBI v1.8.1(QEMU 내장), U-Boot v2024.10, Linux v6.12.59, xv6-riscv `06aad25`. 화면 그림은 모두 tmux 창에서 실제로 실행한 뒤 `tmux capture-pane`으로 받은 글자를 창 모양으로 그린 것이다. 긴 출력은 줄을 골라 냈고, 건너뛴 자리는 `...`로 표시했다. 밑줄 친 소스 위치를 누르면 GitHub의 그 줄이 열린다(태그 고정).

## 1. 부트로더가 메우는 간격

전원을 켠 직후의 기계와 커널이 시작할 때 기대하는 기계는 다르다. 부트로더의 일은 앞의 상태를 뒤의 상태로 바꾸는 것이다.

| 항목 | 전원 직후 | 커널이 기대하는 것 |
|---|---|---|
| 실행 위치 | 리셋 벡터(작은 ROM)의 명령어 하나 | 커널 이미지의 첫 명령어 |
| 메모리 | DRAM 컨트롤러가 꺼져 있을 수 있다(실제 보드) | DRAM이 동작하고 크기를 안다 |
| 커널 이미지 | 플래시, eMMC, SD, 네트워크 너머에 있다 | 메모리의 정해진 주소에 올라와 있다 |
| 권한 모드 | 가장 높은 모드(RISC-V는 M-mode) | 커널용 모드(S-mode), M-mode는 펌웨어가 지킨다 |
| 하드웨어 정보 | 없다 | 디바이스트리 주소(`a1`), 부팅 코어 번호(`a0`) |
| 커널 인자 | 없다 | `bootargs`(콘솔, 루트 파일시스템 등) |

PC에서는 이 일을 BIOS나 UEFI 펌웨어와 GRUB가 나눠 하고, 임베디드 보드에서는 보통 **BootROM → SPL → U-Boot**가 한다. RISC-V에는 여기에 **SBI 펌웨어**(OpenSBI)가 한 단계 더 끼어든다. M-mode에 남아 커널에게 타이머, 코어 간 인터럽트, 전원 끄기 같은 서비스를 제공하는 상주 펌웨어다.

**핵심** 부트로더는 "커널을 메모리에 올리고 점프하는 프로그램"이다. 그 전에 메모리를 살리고, 커널이 알아야 할 정보를 정해진 자리에 놓는다.

## 2. 실습 환경

U-Boot와 Linux는 공식 소스를 받아 QEMU `virt`용 설정으로 빌드했다. OpenSBI는 QEMU에 들어 있는 것을 그대로 쓴다. 커널이 뜬 뒤 디바이스트리를 사용자 공간에서 읽어 보도록, 작은 `/init` 하나만 든 initramfs를 만들었다.

```
$ make -C u-boot-2024.10 O=build/u-boot CROSS_COMPILE=riscv64-linux-gnu- \
       qemu-riscv64_smode_defconfig all          # -> u-boot.bin (S-mode)
$ make -C linux-6.12.59 O=build/linux ARCH=riscv CROSS_COMPILE=... defconfig Image
$ qemu-system-riscv64 -machine virt -m 128M -smp 3 -nographic \
       -bios default -kernel build/u-boot/u-boot.bin \     # OpenSBI, then U-Boot
       -device loader,file=Image,addr=0x84000000 \          # kernel into RAM
       -device loader,file=initramfs.cpio.gz,addr=0x88000000
```

`-bios default`는 QEMU가 OpenSBI를 `0x80000000`에 올리게 하고, `-kernel`로 준 파일은 OpenSBI의 "다음 단계"가 된다. `-device loader`는 실제 보드라면 U-Boot가 저장장치에서 읽어 왔을 커널과 initramfs를 미리 RAM에 놓아 둔다.

## 3. 부팅 사슬

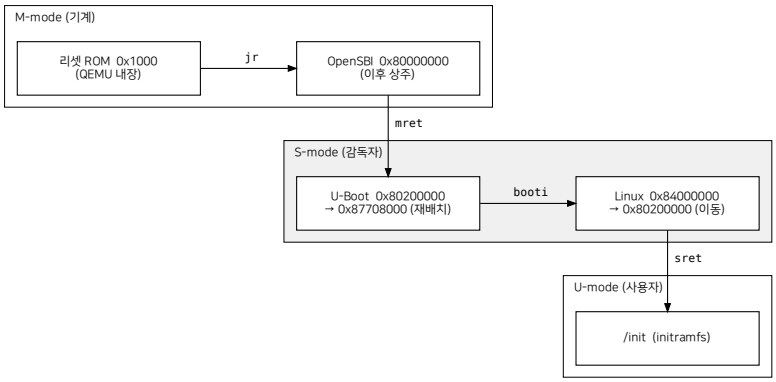



그림 1. QEMU virt의 부팅 사슬. 칸이 권한 모드다. 단계가 넘어갈 때마다 권한이 같거나 낮아진다.

| 단계 | 위치 | 모드 | 하는 일 |
|---|---|---|---|
| 리셋 ROM | `0x1000` | M | 몇 줄짜리 코드. `a0`, `a1`, `a2`를 채우고 `0x80000000`으로 뛴다 |
| OpenSBI | `0x80000000` | M | 코어·타이머·IPI 초기화, PMP로 자기 영역 보호, S-mode로 내려가 다음 단계 실행. 이후 상주 |
| U-Boot | `0x80200000` → `0x87708000` | S | 자기를 RAM 끝으로 재배치, 장치 초기화, 환경 변수, 커널을 읽고 DTB를 고쳐 점프 |
| Linux | `0x84000000` → `0x80200000` | S | 넘겨받은 DTB로 하드웨어를 알아내고 드라이버를 붙인다. 첫 사용자 프로세스 실행 |

실제 보드라면 OpenSBI 앞에 **SPL**(Secondary Program Loader)이 하나 더 있다. BootROM은 칩 안의 작은 SRAM만 쓸 수 있어서, SRAM에 들어갈 만큼 작은 SPL이 먼저 DRAM 컨트롤러를 깨우고 그다음 단계를 DRAM에 올린다. QEMU는 RAM이 처음부터 살아 있으므로 SPL 없이 시작한다.

## 4. 실험 1: OpenSBI만 돌리면

다음 단계 없이 OpenSBI만 띄웠다.

OpenSBI 단독 (커널 없음)

```
$ qemu-system-riscv64 -machine virt -m 128M -smp 3 -nographic -bios default
...
Platform Name               : riscv-virtio,qemu
Platform Features           : medeleg
Platform HART Count         : 3
...
Platform Console Device     : uart8250
...
Firmware Base               : 0x80000000
Firmware Size               : 345 KB
...
Domain0 Next Address        : 0x0000000000000000
Domain0 Next Arg1           : 0x0000000087e00000
Domain0 Next Mode           : S-mode
...
Boot HART ID                : 1
```

그림 2. OpenSBI 단독. 다음 단계 주소가 0이라 여기서 멈춘다.

- `Platform Name : riscv-virtio,qemu`는 OpenSBI가 지어낸 이름이 아니다. 디바이스트리의 `model` 속성을 읽은 것이다. 펌웨어부터 이미 디바이스트리로 보드를 알아낸다.
- `Firmware Base 0x80000000, Size 345 KB`. OpenSBI는 RAM 맨 앞을 차지하고, PMP로 S-mode가 이 영역을 못 건드리게 막는다 (`Domain0 Region00, 01`의 `S/U: ()`).
- `Domain0 Next Address : 0x0`, `Next Arg1 : 0x87e00000`, `Next Mode : S-mode`. 넘겨줄 곳은 비었지만, 넘겨줄 인자(디바이스트리 주소)와 모드는 이미 정해져 있다.

## 5. 실험 2: 단계 사이에 넘기는 것

U-Boot를 다음 단계로 주고, gdb로 U-Boot의 첫 명령어(`0x80200000`)에 중단점을 걸었다. OpenSBI가 `mret`으로 S-mode에 내려온 바로 그 순간이다.

gdb · OpenSBI → U-Boot

```
(gdb) break *0x80200000
Breakpoint 1 at 0x80200000
(gdb) continue
Continuing.
[Switching to Thread 1.2]

Thread 2 hit Breakpoint 1, 0x0000000080200000 in ?? ()
(gdb) info registers a0 a1 a2 pc
a0             0x1      1
a1             0x87e00000       2279604224
a2             0x0      0
pc             0x80200000       0x80200000
(gdb) x/4bx $a1
0x87e00000:     0xd0    0x0d    0xfe    0xed
(gdb) x/2wx $a1+4
```

그림 3. OpenSBI → U-Boot 경계의 레지스터. `a1`이 가리키는 곳의 첫 4바이트가 DTB 매직 넘버다.

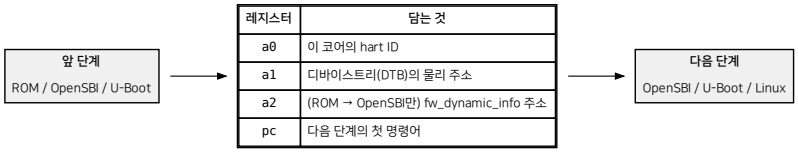



그림 4. RISC-V 부팅 단계 사이의 약속. 리셋 ROM부터 Linux까지 같은 규칙을 따른다.

- `a0 = 1`: 이번에는 hart 1이 부팅 코어가 됐다(OpenSBI 출력의 `Boot HART ID : 1`과 같다). 0번이 늘 먼저인 것은 아니다.
- `a1 = 0x87e00000`: 디바이스트리 주소. 첫 바이트가 `d0 0d fe ed`, 즉 FDT 매직 `0xd00dfeed`다.
- 다음 워드를 32비트로 읽으면 `0x572c0000`이다. DTB는 빅엔디언이라 바이트를 뒤집으면 `0x00002c57` = 11351바이트, DTB 전체 크기다. QEMU가 만든 원본(7108바이트)보다 크다. OpenSBI가 넘기기 전에 노드를 더 써 넣었기 때문이다(디바이스트리 문서 5절).

받는 쪽 코드도 이 약속대로 쓰여 있다. U-Boot는 첫 명령어 근처에서 `a0`을 `tp`에, `a1`을 `s1`에 보관한다 (`arch/riscv/cpu/start.S:50--51`). Linux는 `a0`을 `boot_cpu_hartid`에 저장하고 (`arch/riscv/kernel/head.S:292--294`), `a1`을 `a0`로 옮겨 `setup_vm(dtb_pa)`를 부른다 (`head.S:306--311`).

**핵심** 단계와 단계 사이에 오가는 것은 레지스터 몇 개뿐이다. 그중 가장 중요한 것이 `a1`, 디바이스트리의 주소다.

## 6. U-Boot 안에서




그림 5. U-Boot의 실행 흐름. 재배치 전(`board_init_f`)과 후(`board_init_r`)로 나뉜다.

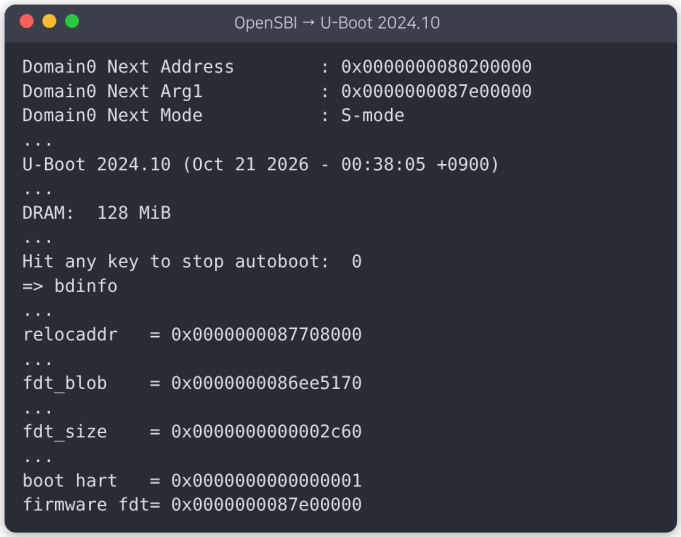



그림 6. U-Boot 시작과 `bdinfo`. 자동 부팅을 멈추고 명령 프롬프트(`=>`)로 들어갔다.

## 6.1 재배치

U-Boot는 `0x80200000`에서 시작하지만 `bdinfo`의 `relocaddr`은 `0x87708000`이다. 처음 실행된 위치에서 RAM 크기를 알아낸 뒤(`DRAM: 128 MiB`), 자기 자신을 RAM 끝으로 복사하고 거기서 다시 시작한다 (`common/board_f.c:795`). 앞쪽 RAM을 커널과 initramfs에 통째로 비워 주려는 것이다. `lmb_dump_all`의 예약 구간 `[0x85ee1000-0x87ffffff]`이 재배치된 U-Boot와 그 스택, 힙, 디바이스트리 사본 자리다. 같은 이유로 디바이스트리도 `0x87e00000`에서 `fdt_blob = 0x86ee5170`으로 옮겨졌다.

## 6.2 환경 변수와 자동 부팅

```
=> printenv bootcmd bootdelay
bootcmd=run distro_bootcmd
bootdelay=2
```

`bootdelay`초 동안 키 입력이 없으면 `bootcmd`를 실행한다. `distro_bootcmd`는 virtio 디스크, 네트워크 같은 장치를 차례로 뒤져 부팅 스크립트나 커널을 찾는다. 실제 제품은 이 변수를 "플래시의 정해진 위치에서 커널을 읽어 `booti`"처럼 짧게 고쳐 둔다.

## 6.3 U-Boot도 디바이스트리로 장치를 안다

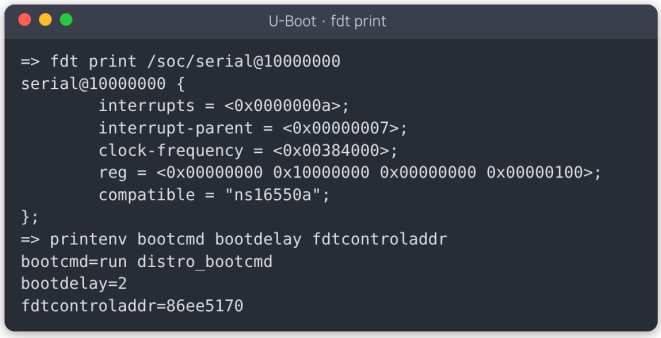



그림 7. U-Boot의 `fdt` 명령으로 본 직렬 포트 노드.

U-Boot의 드라이버 모델도 이 노드의 `compatible = "ns16550a"`와 `reg`를 보고 직렬 포트 드라이버를 붙였다. `bdinfo`의 `serial addr = 0x10000000`이 그 결과다.

## 7. 실험 3: U-Boot에서 Linux로

자동 부팅을 멈춘 프롬프트에서 커널 인자를 정하고 `booti`로 넘겼다. 세 인자는 커널 주소, initramfs의 주소와 크기, 디바이스트리 주소다.

```
=> setenv bootargs "console=ttyS0 earlycon"
=> booti 0x84000000 0x85800000:0x54816 $fdtcontroladdr
```

U-Boot → Linux (booti)

```
=> booti 0x84000000 0x85800000:0x54816 $fdtcontroladdr
Moving Image from 0x84000000 to 0x80200000, end=81796000
## Flattened Device Tree blob at 86ee5170
   Booting using the fdt blob at 0x86ee5170
Working FDT set to 86ee5170
   Using Device Tree in place at 0000000086ee5170, end 0000000086eeadc6
Working FDT set to 86ee5170
No RNG device

Starting kernel ...
...
[    0.000000] Linux version 6.12.59 ...
...
[    0.000000] Machine model: riscv-virtio,qemu
...
[    0.000000] earlycon: ns16550a0 at MMIO 0x0000000010000000 (options '')
...
[    0.000000] OF: reserved mem: 0x0000000080000000..0x000000008003ffff (256 KiB) mmode_resv1@80000000
[    0.000000] OF: reserved mem: 0x0000000080040000..0x000000008005ffff (128 KiB) mmode_resv0@80040000
...
[    0.000000] Kernel command line: console=ttyS0 earlycon
...
[    0.458802] Unpacking initramfs...
...
[    0.878892] 10000000.serial: ttyS0 at MMIO 0x10000000 (irq = 12, base_baud = 230400) is a 16550A
...
[    1.139955] Run /init as init process
```

그림 8. `booti`에서 `/init`까지. 커널 로그는 디바이스트리와 관련된 줄만 골랐다.

### 7.1 booti가 한 일

- **커널 옮기기**: `Moving Image from 0x84000000 to 0x80200000`. RISC-V 커널 Image 헤더에는 "RAM 시작에서 얼마 떨어진 곳에서 돌아야 하는가"가 적혀 있고, `booti`가 그 자리로 옮긴다 (`cmd/booti.c:81`, 헤더 해석은 `arch/riscv/lib/image.c`). 그 자리는 원래 U-Boot가 처음 실행되던 곳이다. U-Boot가 RAM 끝으로 재배치한 덕에 비어 있다.
- **디바이스트리 고치기**: 넘기기 직전 `/chosen`에 `bootargs`와 initramfs 위치(`linux,initrd-start/end`)를 써 넣는다 (`boot/image-fdt.c:598`의 `fdt_chosen`, `:650`의 `fdt_initrd`). 아래 두 그림이 그 전과 후다.
- **점프**: `a0`에 hart ID, `a1`에 디바이스트리 주소(`0x86ee5170`)를 넣고 커널의 첫 명령어로 뛴다. 실험 2와 같은 약속이다.

U-Boot · booti 전의 /chosen

```
=> fdt print /chosen
chosen {
        stdout-path = "/soc/serial@10000000";
        rng-seed = <...>;
};
```

그림 9. `booti` 전의 `/chosen`. `bootargs`가 없다.

Linux · /init

```
=== init: device tree seen from user space ===
/proc/device-tree/model                            riscv-virtio,qemu
/proc/device-tree/chosen/bootargs                  console=ttyS0 earlycon
/proc/device-tree/memory@80000000/device_type      memory
/proc/device-tree/soc/serial@10000000/compatible ns16550a
platform devices -> bound driver
  10008000.virtio_mmio          -> (no driver)
...
  poweroff                      -> syscon-poweroff
...
  soc                           -> simple-pm-bus
...
  2000000.clint                 -> (no driver)
...
  101000.rtc                    -> goldfish_rtc
...
  c000000.interrupt-controller -> riscv-plic
...
  10000000.serial               -> of_serial
...
[    1.215284] reboot: Power down
```

그림 10. 커널이 뜬 뒤 `/init`이 읽은 디바이스트리. `bootargs`가 들어 있다. U-Boot가 써 넣은 것이다.

## 7.2 넘겨받은 커널이 처음 하는 일

커널 로그의 앞부분은 거의 다 디바이스트리를 읽은 결과다.

- `Machine model: riscv-virtio,qemu`: 루트 노드의 `model`(drivers/of/fdt.c:747).
- `earlycon: ns16550a0 at MMIO 0x10000000`: 드라이버가 뜨기 전에 `/chosen/stdout-path`가 가리키는 직렬 포트로 로그를 낸다.
- `OF: reserved mem: ... mmode_resv1@80000000`: OpenSBI가 디바이스트리에 써 둔 "여기는 쓰지 마라" 구간. 커널은 이 384 KB를 피한다.
- `Kernel command line`: `/chosen/bootargs`. `Unpacking initramfs`: `/chosen/linux,initrd-start`.

| 주소 | 내용 | 누가 놓았나 |
|---|---|---|
| `0x87e00000` | DTB 원본 (OpenSBI가 고친 것) | OpenSBI |
| `0x87708000` | U-Boot (재배치 후) | U-Boot 자신 |
| `0x86ee5170` | DTB 사본 → 커널에 넘김 | U-Boot |
| `0x85ee1000` | U-Boot 스택·힙 (예약 구간 시작) | U-Boot |
| `0x85800000` | initramfs (336 KB) | QEMU loader |
| `0x84000000` | 커널 Image 적재 위치 (22 MB) | QEMU loader |
| `0x80200000` | 커널 실행 위치 (booti가 옮김) | U-Boot |
| `0x80000000` | OpenSBI (384 KB, S-mode 접근 금지) | QEMU |

그림 11. `booti` 직전 RAM 128 MB의 배치. 모두 이번 실험의 출력에서 읽은 주소다.

**핵심** `booti`는 커널을 제자리로 옮기고, 디바이스트리의 `/chosen`에 커널 인자와 initramfs 위치를 적고, `a1`에 그 주소를 실어 점프한다.

## 8. 비교: xv6에는 부트로더가 없다

xv6는 QEMU를 `-bios none`으로 띄운다. OpenSBI도 U-Boot도 없이 리셋 ROM이 바로 `0x80000000`의 xv6 커널로 뛴다. 그래서 xv6 커널은 부트로더가 할 일을 스스로 한다.

| 부트로더의 일 | Linux (이 문서) | xv6 |
|---|---|---|
| M-mode 설정, S-mode로 내려가기 | OpenSBI | `start.c`가 직접 하고 `mret` |
| 타이머·IPI 같은 M-mode 서비스 | OpenSBI가 SBI 호출로 제공 | 직접 CSR을 다룬다 |
| 커널을 메모리에 올리기 | U-Boot | QEMU `-kernel`이 대신 |
| 하드웨어 정보 | DTB 주소를 `a1`로 받는다 | 받지 않는다. `memlayout.h`에 주소가 상수로 있다 |

xv6는 한 보드(QEMU virt)만 다루는 교육용 커널이라 이렇게 해도 된다. 여러 보드에서 같은 커널 바이너리를 돌리려면 펌웨어·부트로더·디바이스트리라는 층이 필요해진다.

## 9. 실제 보드에서 더해지는 것

- **SPL과 DRAM 초기화**: BootROM이 읽을 수 있는 크기와 위치(플래시의 앞부분)가 정해져 있다. DRAM 타이밍 설정은 보드마다 다르다.
- **저장장치에서 읽기**: NOR/NAND 플래시, eMMC, SD에서 커널, DTB, initramfs를 읽는다. 플래시를 영역(파티션)으로 나눠 단계마다 자리를 정한다.
- **FIT 이미지**: 커널, DTB, initramfs와 그 해시·서명을 한 파일에 묶는 U-Boot 형식(`.itb`). `bootm`으로 부팅한다.
- **보안 부팅**: 각 단계가 다음 단계의 서명을 확인한 뒤에만 점프한다. 첫 단계(BootROM)를 믿는 것에서 신뢰가 시작된다.
- **업데이트와 복구**: A/B 영역, 부팅 실패 횟수 세기, 네트워크(TFTP)로 이미지 받기 같은 기능을 부트로더가 맡는다.

## 10. 정리

1. 부트로더는 전원 직후의 기계를 커널이 기대하는 상태로 바꾸고, 커널을 메모리에 올려 점프한다.
2. RISC-V QEMU virt의 사슬은 리셋 ROM → OpenSBI(M) → U-Boot(S) → Linux(S) → `/init`(U)이다.
3. OpenSBI는 M-mode에 상주하며 커널에 SBI 서비스를 준다. 실제 보드는 그 앞에 DRAM을 깨우는 SPL이 있다.
4. 단계 사이에는 `a0`(hart ID)과 `a1`(디바이스트리 주소)을 넘긴다. 실험에서 `a1 = 0x87e00000`, 매직 `0xd00dfeed`를 확인했다.
5. U-Boot는 RAM 끝으로 재배치하고, 환경 변수(`bootcmd`)에 따라 커널을 찾고, `booti`로 넘기기 전에 디바이스트리의 `/chosen`을 고친다.
6. xv6는 부트로더 없이 이 일을 커널이 직접 하고, 하드웨어 주소를 소스에 박아 둔다.